

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий інститут прикладного системного аналізу
Кафедра системного проектування

Звіт

про виконання лабораторної роботи №1
з дисципліни «Інтелектуальні сервіс-орієнтовані системи»

Виконав:
студент IV курсу, групи КН-61мн
Терещенко Олексій Ігорович

Київ – 2026

ТЕОРЕТИЧНІ ВІДОМОСТІ

Предметна область за варіантом 5 - хмарна інформаційна система обліку часу роботи за комп’ютером.

Event Storming описує процес через команди, доменні події та реакції на них. Подія фіксує факт, що вже відбувся, а команда виражає намір змінити стан. Domain-Driven Design пов’язує програмну модель із правилами предметної області; bounded context задає межі, у яких терміни та правила мають однозначне значення. Сутність зберігає ідентичність при зміні властивостей, тоді як об’єкт-значення визначається сукупністю своїх значень.

Мета роботи - визначити бізнес-процеси системи обліку робочого часу та розподілити їх між незалежними сервісами. Застосувати Event Storming і Domain-Driven Design для визначення контекстів, доменних об’єктів та способів взаємодії.

Постановка задачі

Система призначена для збереження робочих сесій користувача та перегляду накопиченого часу. Користувач передає тривалість завершеної роботи, а система перевіряє дані, зберігає запис і включає його до звіту. Наприклад, сесії тривалістю 1500 і 900 секунд утворюють підсумок 2400 секунд, або 40 хвилин.

Для запису потрібні ідентифікатор користувача та додатна ціла тривалість у секундах. Профіль містить допустимий ліміт. У прийнятій моделі поле daily_limit=28800 обмежує одну сесію; підсумок звіту накопичується за весь час. Автоматичне визначення активності комп’ютера та облік календарного дня виходять за межі цього сценарію.

ХІД РОБОТИ

Event Storming

Процес розглянуто як послідовність команди, зміни стану та подальшої реакції системи. Команда «Записати сесію» виражає намір користувача. Після перевірки профілю й тривалості створюється WorkSession та виникає факт SessionRecorded. На цей факт реагує політика оновлення статистики: Reporting додає сесію до власної проєкції та перераховує звіт.

На дошці використано такі типи елементів.

Тип	**Приклад**	**Позначення**
Актор	Користувач	Жовтий
Команда	Записати сесію	Синій
Агрегат	WorkSession	Світло-сірий
Подія	Сесію записано	Помаранчевий
Політика	Оновити статистику	Фіолетовий
Проєкція	TimeReport	Зелений

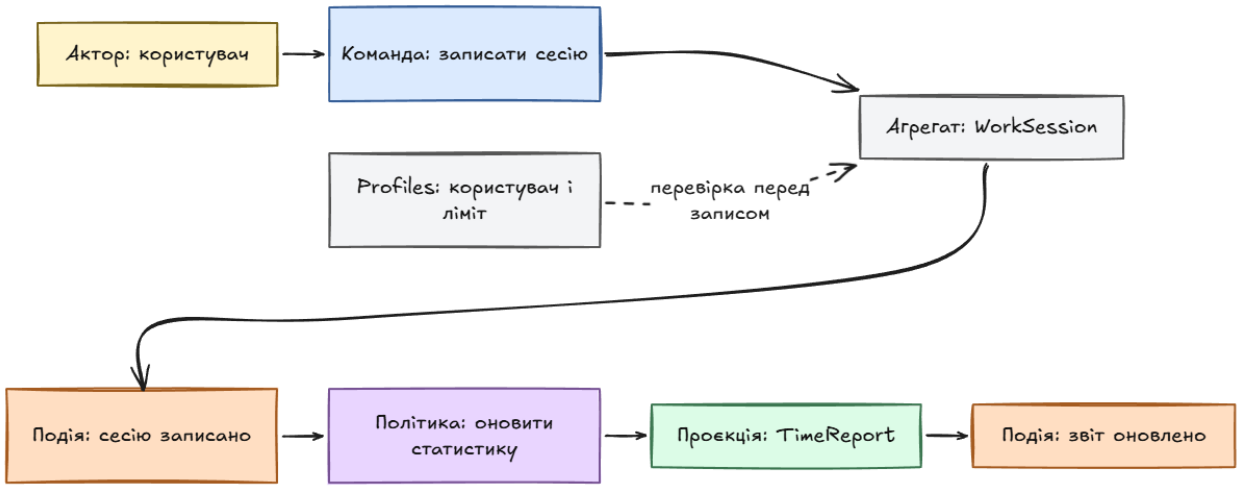


Рисунок 1 – дошка Event Storming.

Під час аналізу процесу визначено такі проблемні ситуації.

Ситуація	**Рішення**
Від'ємна тривалість або перевищення ліміту	Tracking відхиляє команду до збереження
Профіль користувача відсутній	Повертається помилка, сесія не створюється

Ситуація	Рішення
Profiles тимчасово недоступний	Виклик обмежується timeout; клієнт отримує контрольовану помилку
Подія доставлена повторно	Reporting перевіряє id та не враховує сесію двічі
Звіт запитано до обробки події	Новий запис з’явиться після оновлення проєкції
RabbitMQ недоступний після запису сесії	Виникає ризик втрати події; для надійної реалізації передбачено transactional outbox

Обмежені контексти та карта взаємодії

Предметну область поділено на три bounded contexts. Межі проведено за відповідальністю та власністю даних.

Контекст	Відповідальність	Власні дані
Profiles	Профілі користувачів та допустимі ліміти	Ідентифікатор, ім’я, ліміт
Tracking	Перевірка та збереження робочих сесій	Первинні записи часу
Reporting	Побудова та читання статистики	Звітна проєкція

Tracking є основним піддоменом: він реалізує облік робочого часу. Reporting підтримує цей процес, перетворюючи записи на підсумок. Profiles надає відомості для перевірки користувача.

Profiles постачає Tracking відкритий HTTP-контракт. На карті це відношення Open Host Service, а JSON-подання профілю виконує роль Published Language. Tracking використовує відповідь для перевірки, зберігаючи всередині сесії лише user_id.

Tracking постачає Reporting події про записані сесії. Поля узгоджуються відповідно до потреб звітності, що відображено відношенням Customer-Supplier. Формат session.created є інтеграційним контрактом. Reporting не залежить від структури таблиць Tracking.

Прямий зв’язок Profiles і Reporting для підрахунку часу не потрібний. На Context Map стрілки спрямовані від постачальника даних до споживача. На схемі компонентів HTTP-стрілки показують напрямок мережевого запиту.



Рисунок 2 – карта контекстів Profiles Tracking Reporting.

Доменна модель

Контекст Profiles

UserProfile має сталий ідентифікатор і є коренем агрегата профілю. Зміна імені не змінює ідентичність користувача. Агрегат контролює допустимість власних значень.

Об'єкт	**Тип**	**Властивості та правила**
UserProfile	Entity і Aggregate Root	Ідентичність визначається user_id; профіль зберігається як одне ціле
UserId	Value Object	Додатний ідентифікатор профілю
DisplayName	Value Object	Непорожнє ім'я користувача
TimeLimitSeconds	Value Object	Додатна допустима тривалість сесії

Пошук і збереження профілів виконує ProfileRepository. Прикладний сервіс приймає запит, звертається до моделі та репозиторію. Правила коректності профілю залишаються в доменній моделі.

Контекст Tracking

Коренем агрегата є WorkSession. До його меж належать ідентифікатор, посилання на користувача, тривалість та час запису. Один агрегат зберігається в одній локальній транзакції.

Об'єкт	**Тип**	**Властивості та правила**
WorkSession	Entity і Aggregate Root	Унікальна сесія з допустимою тривалістю
SessionId	Value Object	Пов'язує сесію з інтеграційною подією
UserId	Value Object	Посилання на профіль користувача
DurationSeconds	Value Object	Ціле число більше нуля, у межах ліміту
RecordedAt	Value Object	Час приймання запису в UTC

Дві сесії з однаковою тривалістю є різними сутностями, якщо мають різні SessionId. Два значення DurationSeconds, що дорівнюють 1500, рівні за значенням.

Профіль не включається до агрегата сесії. Його наявність перевіряє прикладний обробник перед збереженням. Так межа локальної транзакції не поширюється на два сервіси. Після запису виникає доменна подія SessionRecorded; для передачі між сервісами використовується інтеграційне подання session.created.

Контекст Reporting

TimeReport є моделлю читання. Вона містить підсумок робочого часу користувача та формується з отриманих подій. Одна сесія впливає на підсумок не більше одного разу. Позначка обробленого id й оновлення звітної проєкції виконуються в одній транзакції.

Загальна мова

Термін	**Значення**
Профіль користувача	Дані, що ідентифікують користувача та задають ліміт
Робоча сесія	Запис завершеної роботи за комп’ютером
Тривалість	Додатна кількість секунд роботи
Записати сесію	Команда перевірити та зберегти завершену сесію
Сесію записано	Факт збереження первинного запису
Звіт	Підсумкове подання часу користувача
Проєкція	Похідні дані, підготовлені для читання
Ідемпотентна обробка	Повторне повідомлення з тим самим id не змінює підсумок удруге

Компоненти системи

Кожному контексту відповідає окремий сервіс. API Gateway надає єдину точку входу. Перевірка профілю відбувається синхронно через HTTP, а оновлення звітності — асинхронно через RabbitMQ.

Компонент	**Призначення**	**Інтерфейс або сховище**
API Gateway	Маршрутизація запитів	HTTP JSON

Компонент	Призначення	Інтерфейс або сховище
Profiles	Читання профілю	GET /users/{id}; власна БД
Tracking	Приймання та збереження сесії	POST /sessions; власна БД
Reporting API	Читання звіту	GET /reports/{user_id}
Reporting worker	Обробка події й оновлення проєкції	AMQP; БД Reporting
RabbitMQ	Доставка подій	Exchange sessions; queue reports

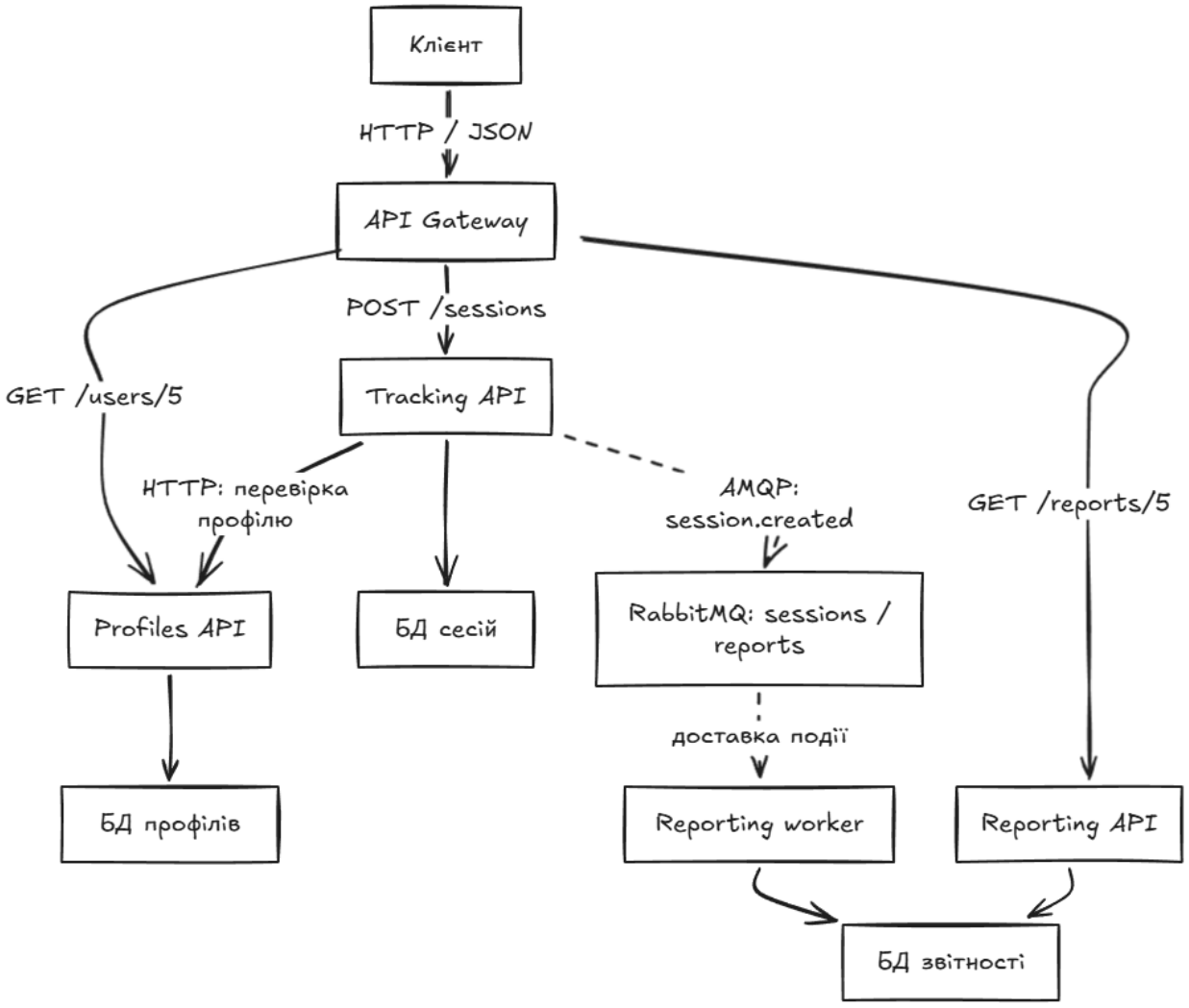


Рисунок 3 – схема програмних та інфраструктурних компонентів.

Кожен сервіс працює лише зі своєю базою. Reporting API та worker належать одному контексту й використовують його звітне сховище. Первинним джерелом відомостей про сесії залишається Tracking.

Приклад команди запису: user_id=5, duration_seconds=1500. Подія session.created містить id сесії, user_id та duration_seconds. Власність даних і

контракт повідомлення дозволяють змінювати внутрішню реалізацію сервісів окремо.

Послідовність викликів

Успішне приймання сесії

Клієнт передає команду через Gateway до Tracking. Tracking запитує профіль, перевіряє тривалість і зберігає WorkSession у власній базі. Після цього публікується session.created, а клієнт отримує підтвердження приймання.

Reporting отримує подію та в локальній транзакції перевіряє id й оновлює проєкцію. ACK надсилається після COMMIT. Окремим HTTP-запитом клієнт отримує поточний звіт. До завершення асинхронної обробки звіт може не містити останню сесію.

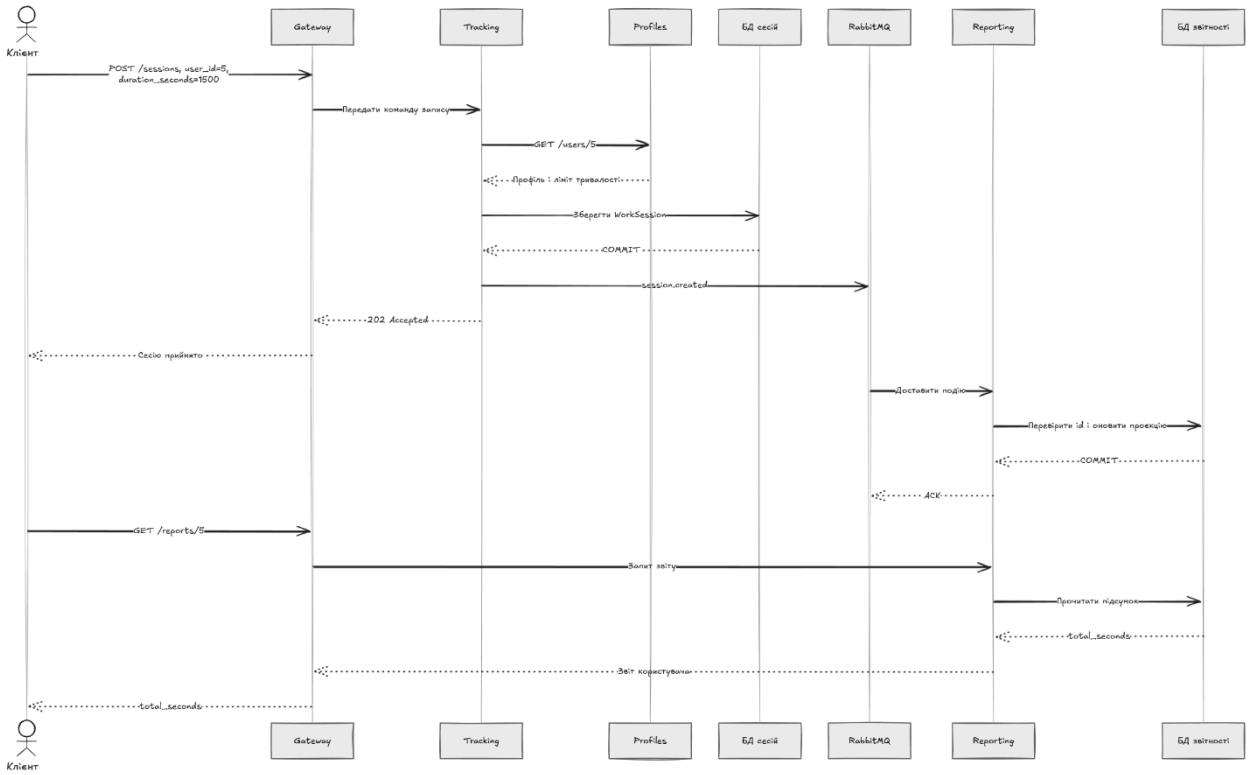


Рисунок 4 – діаграма послідовності успішного сценарію.

Відмова перевірки профілю

Якщо Profiles повертає 404, Tracking відхиляє команду. Якщо Profiles не відповідає до завершення timeout, Tracking повертає помилку недоступності. В обох випадках запис сесії та публікація події не відбуваються.

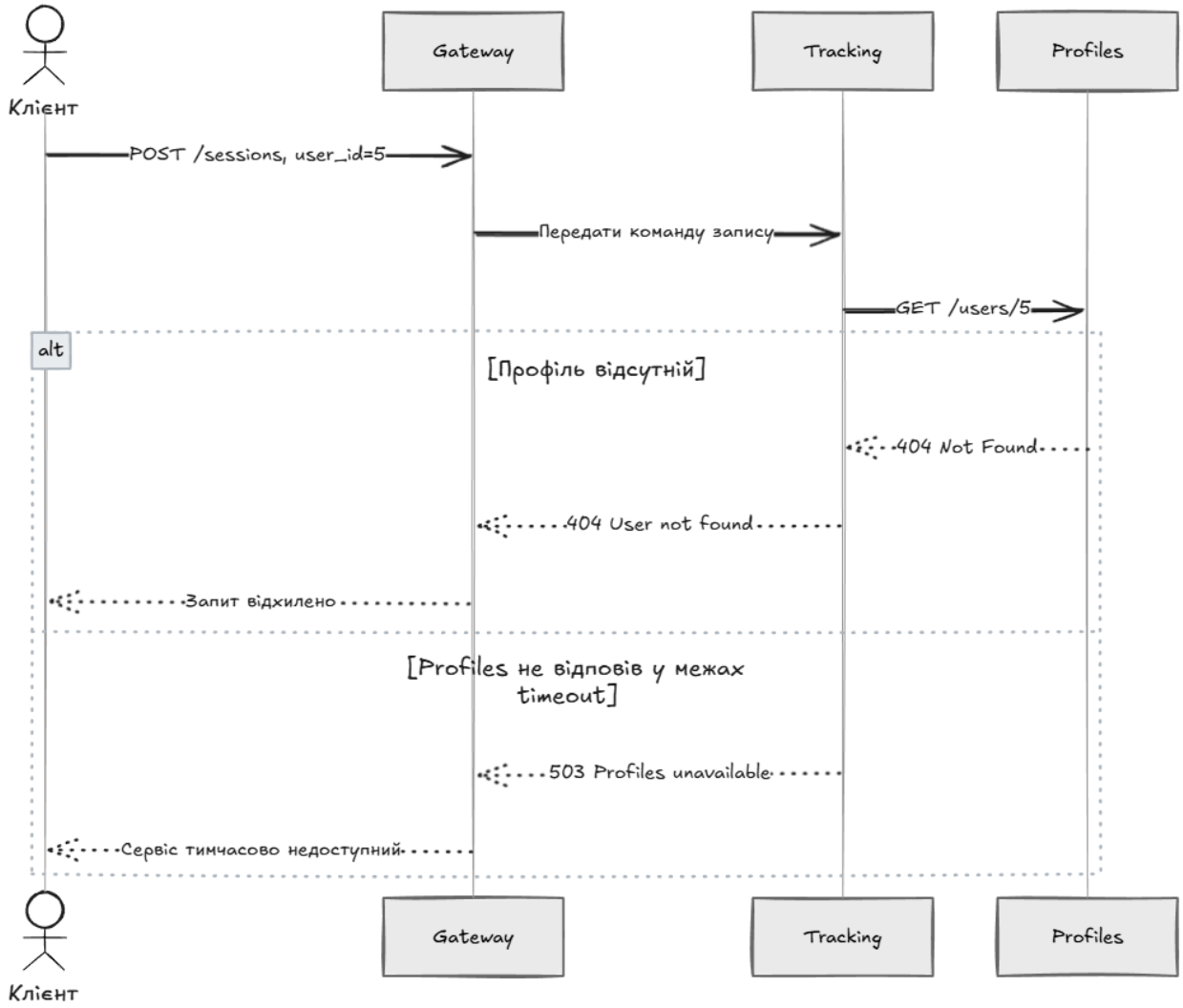


Рисунок 5 – діаграма послідовності відмови Profiles.

Результати проєктування

Поділ за бізнес-відповідальністю дозволяє окремо змінювати профілі, облік сесій та звітність. Власні бази й вузькі контракти обмежують залежності між сервісами. Асинхронна проєкція відокремлює швидкість приймання сесій від швидкості побудови звітів.

Водночас з’являються часткові відмови та тимчасова неузгодженість даних. Перевірка профілю залежить від доступності Profiles. Повторна

доставка вимагає ідемпотентного consumer. Незалежні COMMIT у Tracking та публікація в RabbitMQ утворюють проміжок, у якому подія може бути втрачена. Для подальшої реалізації цей ризик усувається transactional outbox.

Для невеликого застосунку модульний моноліт був би простішим у запуску й підтримці. Обрана декомпозиція дає можливість розглянути контракти, власність даних та мережеву взаємодію. Поділ до окремого сервісу на кожну операцію збільшив би витрати без корисної бізнес-автономії.

ВИСНОВКИ

Межі сервісів доцільно визначати за бізнес-правилами та власністю даних. Профіль, запис робочої сесії та побудова звіту мають різні причини для зміни, тому їх розділення на Profiles, Tracking і Reporting зменшує залежність моделей одна від одної. Ідентифікатор користувача пов’язує контексти без перенесення всього профілю до агрегата сесії.

Асинхронне оновлення звіту дозволяє приймати сесії незалежно від швидкості обробки статистики, проте створює проміжок тимчасової неузгодженості. Звідси випливає потреба розрізняти приймання команди та появу її результату у звіті. Повторна доставка також вимагає перевірки ідентифікатора події, інакше одна сесія може збільшити суму двічі.

Аналіз сценаріїв відмов показує, що окремі бази даних самі по собі не забезпечують надійності обміну. Між збереженням сесії та публікацією повідомлення залишається ризик втрати події, який потребує transactional outbox. Для невеликого застосунку така декомпозиція додає витрати на мережеву взаємодію; її користь полягає в автономності обліку та звітності, а не в самій кількості сервісів.